CS316

INTRODUCTION TO AI AND DATA SCIENCE

Topic 3
Feature engineering and dimensionality reduction

→adding/removing unnecesary features (too much features would take longer to train)

LECTURE 1

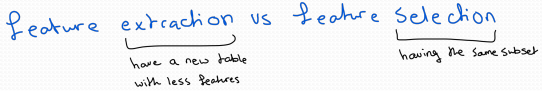
NumPy Library, similarity measure and Mathematical foundations *(Vectors, Matrices, Dot Product Normalization and Cosine Similarity)*

Instructor: Dr. Heyfa Ammar

Outline of this lecture

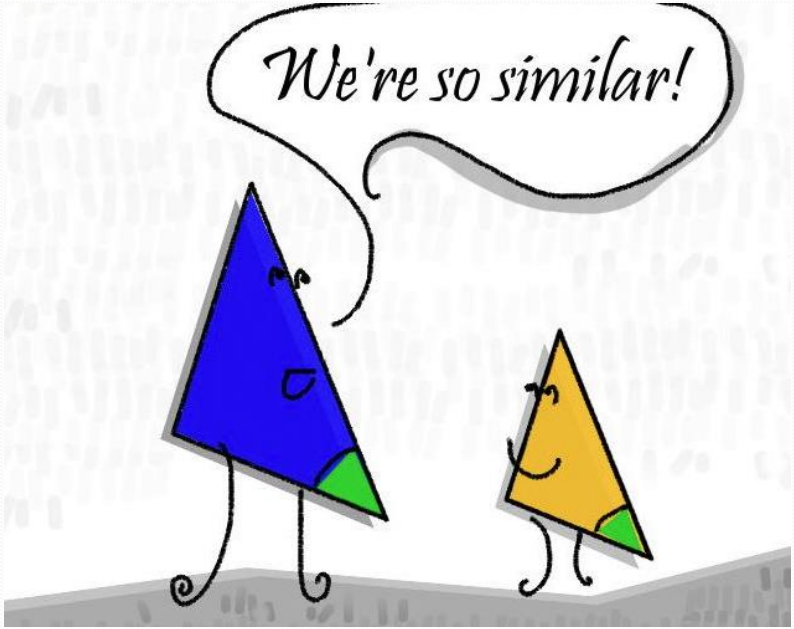
In this lecture, we will discuss the following:

- Why similarity assessment is important and how it works?
- Object embeddings
- Vector representation
- Numpy Library
- Dot product and similarity assessment
- Practice activity

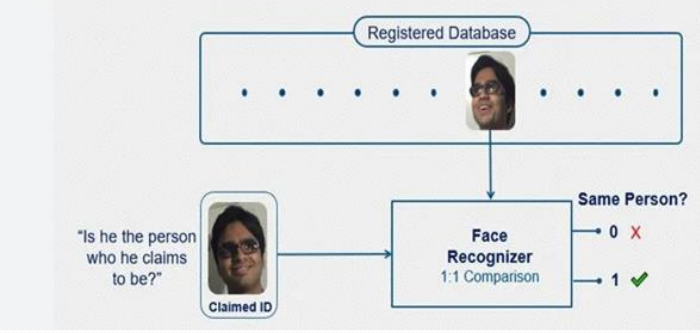


Similarity assessment

Why do we need to measure similarity?



Similarity assessment

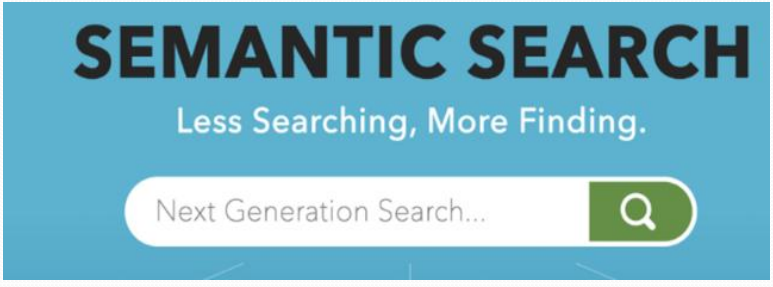


Face recognition



Voice recognition

extract characteristic of the person and convert the image into vectors where we can compare similarities





Similarity assessment : How it works?



```
[[0.1608, 0.1843, 0.1843,  ..., 0.2039, 0.1804, 0.1922],
 [0.1725, 0.1686, 0.1686,  ..., 0.1961, 0.1922, 0.1804],
 [0.1882, 0.1843, 0.1804,  ..., 0.1961, 0.1882, 0.1843],
 ...,
 [0.1686, 0.1725, 0.1725,  ..., 0.1412, 0.1451, 0.1373],
 [0.1765, 0.1725, 0.1765,  ..., 0.1451, 0.1529, 0.1294],
 [0.1804, 0.1608, 0.1843,  ..., 0.1373, 0.1451, 0.1490]]
```



```
[[0.1725, 0.1686, 0.1255,  ..., 0.2039, 0.1882, 0.2039],
 [0.1725, 0.1490, 0.1294,  ..., 0.2000, 0.2000, 0.2118],
 [0.1608, 0.1647, 0.1137,  ..., 0.1961, 0.2078, 0.2078],
 ...,
 [0.1529, 0.1451, 0.1529,  ..., 0.1608, 0.1451, 0.1451],
 [0.1608, 0.1608, 0.1490,  ..., 0.1725, 0.1412, 0.1255],
 [0.1569, 0.1412, 0.1647,  ..., 0.1686, 0.1686, 0.1451]]
```



```
[[0.1529, 0.1725, 0.2078,  ..., 0.1843, 0.1882, 0.1843],
 [0.1725, 0.1725, 0.2392,  ..., 0.1843, 0.1765, 0.1804],
 [0.2314, 0.1529, 0.2235,  ..., 0.1804, 0.1804, 0.1725],
 ...,
 [0.1922, 0.1804, 0.1647,  ..., 0.1176, 0.1255, 0.1098],
 [0.1804, 0.1373, 0.1333,  ..., 0.1176, 0.1137, 0.1059],
 [0.1255, 0.1451, 0.1059,  ..., 0.1137, 0.1020, 0.1137]]
```

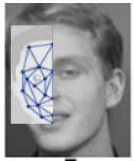
Similarity assessment : How it works?



```
[[0.1608, 0.1843, 0.1843,  ..., 0.2039, 0.1804, 0.1922],
 [0.1725, 0.1686, 0.1686,  ..., 0.1961, 0.1922, 0.1804],
 [0.1882, 0.1843, 0.1804,  ..., 0.1961, 0.1882, 0.1843],
 ...,
 [0.1686, 0.1725, 0.1725,  ..., 0.1412, 0.1451, 0.1373],
 [0.1765, 0.1725, 0.1765,  ..., 0.1451, 0.1529, 0.1294],
 [0.1804, 0.1608, 0.1843,  ..., 0.1373, 0.1451, 0.1490]]
```

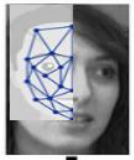


```
[[0.1725, 0.1686, 0.1255,  ..., 0.2039, 0.1882, 0.2039],
 [0.1725, 0.1490, 0.1294,  ..., 0.2000, 0.2000, 0.2118],
 [0.1608, 0.1647, 0.1137,  ..., 0.1961, 0.2078, 0.2078],
 ...,
 [0.1529, 0.1451, 0.1529,  ..., 0.1608, 0.1451, 0.1451],
 [0.1608, 0.1608, 0.1490,  ..., 0.1725, 0.1412, 0.1255],
 [0.1569, 0.1412, 0.1647,  ..., 0.1686, 0.1686, 0.1451]]
```



```
[[0.1529, 0.1725, 0.2078,  ..., 0.1843, 0.1882, 0.1843],
 [0.1725, 0.1725, 0.2392,  ..., 0.1843, 0.1765, 0.1804],
 [0.2314, 0.1529, 0.2235,  ..., 0.1804, 0.1804, 0.1725],
 ...,
 [0.1922, 0.1804, 0.1647,  ..., 0.1176, 0.1255, 0.1098],
 [0.1804, 0.1373, 0.1333,  ..., 0.1176, 0.1137, 0.1059],
 [0.1255, 0.1451, 0.1059,  ..., 0.1137, 0.1020, 0.1137]]
```

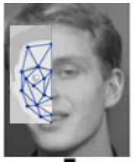
Similarity assessment : How evaluate it?



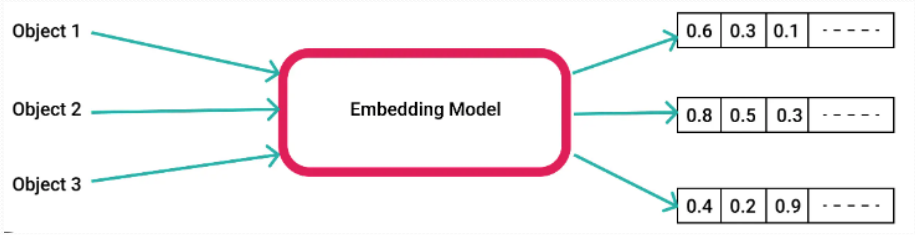
```
[[0.1608, 0.1843, 0.1843,  ..., 0.2039, 0.1804, 0.1922],
 [0.1725, 0.1686, 0.1686,  ..., 0.1961, 0.1922, 0.1804],
 [0.1882, 0.1843, 0.1804,  ..., 0.1961, 0.1882, 0.1843],
 ...,
 [0.1686, 0.1725, 0.1725,  ..., 0.1412, 0.1451, 0.1373],
 [0.1765, 0.1725, 0.1765,  ..., 0.1451, 0.1529, 0.1294],
 [0.1804, 0.1608, 0.1843,  ..., 0.1373, 0.1451, 0.1490]]
```



```
[[0.1725, 0.1686, 0.1255,  ..., 0.2039, 0.1882, 0.2039],
 [0.1725, 0.1490, 0.1294,  ..., 0.2000, 0.2000, 0.2118],
 [0.1608, 0.1647, 0.1137,  ..., 0.1961, 0.2078, 0.2078],
 ...,
 [0.1529, 0.1451, 0.1529,  ..., 0.1608, 0.1451, 0.1451],
 [0.1608, 0.1608, 0.1490,  ..., 0.1725, 0.1412, 0.1255],
 [0.1569, 0.1412, 0.1647,  ..., 0.1686, 0.1686, 0.1451]]
```



```
[[0.1529, 0.1725, 0.2078,  ..., 0.1843, 0.1882, 0.1843],
 [0.1725, 0.1725, 0.2392,  ..., 0.1843, 0.1765, 0.1804],
 [0.2314, 0.1529, 0.2235,  ..., 0.1804, 0.1804, 0.1725],
 ...,
 [0.1922, 0.1804, 0.1647,  ..., 0.1176, 0.1255, 0.1098],
 [0.1804, 0.1373, 0.1333,  ..., 0.1176, 0.1137, 0.1059],
 [0.1255, 0.1451, 0.1059,  ..., 0.1137, 0.1020, 0.1137]]
```

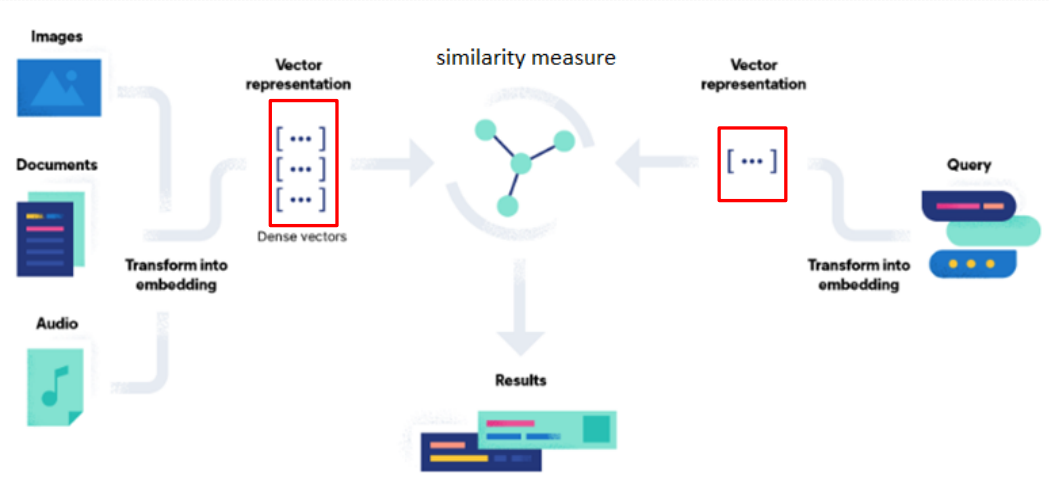


Objects as

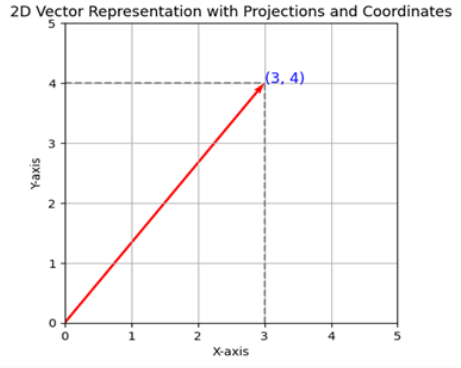
Similarity assessment : How it works?

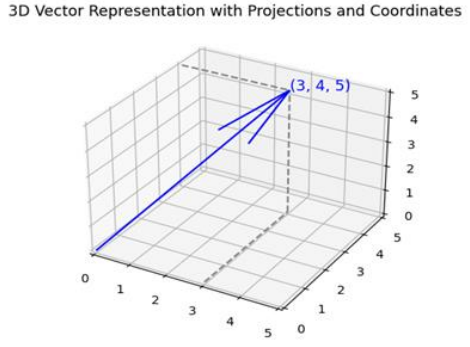






Vector representation





Similar vector have similar direction and similar values

How do we create and manipulate vectors in Python?

Efficiency Comparison: List and NumPy Array

Objective:

- Understand how NumPy arrays enhance performance for numerical operations compared to lists.

Python Code Example:

- **Setup:** Create both a list and a NumPy array with 1,000,000 numerical elements.
- **Operation:** Sum all elements in the data structure.

Key Takeaways:

- NumPy arrays provide significant performance improvements for operations involving large numerical data.
- The efficiency gain is due to NumPy's internal optimizations and C-based processing.
- Using NumPy can lead to more scalable and faster code for data-intensive tasks.

Discussion:

- What implications do these results have for data processing and analysis?
- In what scenarios might you still prefer to use a list over a NumPy array?

Time to sum elements in the list: **0.302939** seconds
Time to sum elements in the NumPy array: **0.009983** seconds

less time in numPy

python Copy code

```python
import timeit
import numpy as np

# Create a large list and a NumPy array with a million elements each
large_list = list(range(1000000))
large_array = np.array(large_list)

# Define functions to sum elements
def sum_list():
    return sum(large_list)

def sum_array():
    return np.sum(large_array)

# Measure the time it takes to sum the elements
list_time = timeit.timeit(sum_list, number=100)
array_time = timeit.timeit(sum_array, number=100)

print(f"Time to sum elements in the list: {list_time:.6f} seconds")
print(f"Time to sum elements in the NumPy array: {array_time:.6f} seconds")
```

What is NumPy?

- **NumPy Overview**
  - A fundamental Python library for numerical computing.
  - Provides support for large, multi-dimensional arrays and matrices.
  - Includes a collection of mathematical functions to operate on these arrays.
- **Why NumPy?**
  - **Performance**: Implemented in C, providing fast execution.
  - **Functionality**: Includes functions for linear algebra, Fourier transform, and random number generation.
  - **Integration**: Forms the basis of most Python-based scientific computing solutions.

```python
import numpy as np
# Creating a simple NumPy array
a = np.array([1, 2, 3, 4, 5])
print(a)
```

Array [1 2 3 4 5]

dtype=int64

Why Use NumPy?

- **Advantages of NumPy**
  - Optimized for numerical operations with N-dimensional array support.
  - Essential for scientific computing with Python.
  - Facilitates complex mathematical functions and operations.
- **Powerful Computing Capabilities**
  - Efficient operations with arrays and matrices.
  - Supports broadcasting and advanced indexing techniques.
  - Integrates C/C++ and Fortran code for high performance.

NumPy Library: Creation of vectors

- **Linear Algebra Concept:**
  - **Definition of Scalars**: Scalars are single numerical values that can represent quantities like length or temperature.
  - **Operations with Scalars**: Common operations include addition, subtraction, multiplication, and division.
- **NumPy Implementation:**
  - **Creating Scalar Variables:**

```python
import numpy as np
x = np.array(5)  # x is a scalar in NumPy
```

  - **Performing Arithmetic Operations with NumPy Scalars:**

```python
y = x - 2    # Subtraction
z = x * 3    # Multiplication
w = x / 2    # Division
print("Subtract 2:", y)
print("Multiply by 3:", z)
print("Divide by 2:", w)
```

Using Scalars in NumPy:

- Scalars in NumPy behave similarly to numbers in basic Python but are optimized for performance when used in array operations.
- They automatically integrate with NumPy's universal functions for array-based computing.

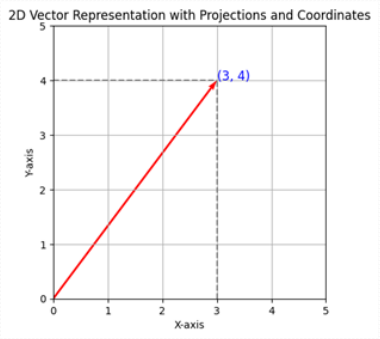
1D Vectors (Arrays)

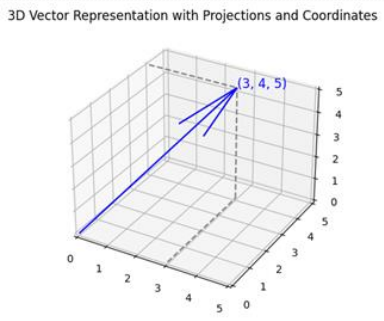
- **Linear Algebra Concept:**
  - **Definition of Vectors**: Vectors are denoted as $\mathbf{v} = [v_1, v_2, \ldots, v_n]$ in mathematics, representing an ordered collection of n elements, each element a coordinate in n-dimensional space.
- **Data Science Application:**
  - **Feature Vectors**: In data science, vectors are used as feature vectors, where each element v_i represents a distinct attribute or feature of a data point, crucial for models in AI and machine learning.
- **NumPy Implementation:**
  - **Representing Vectors Using 1D Arrays:**

```python
import numpy as np
# Example of initializing a feature vector in NumPy
feature_vector = np.array([5, 3, 1, 8])
```

  - **Initializing Vectors:**

```python
# Creating zero and one vectors commonly used in initialization
zero_vector = np.zeros(4)  # Vector of zeros
one_vector = np.ones(4)    # Vector of ones
```






```python
# Define a 2D and a 3D vector
vector_2d = np.array([3, 4])
vector_3d = np.array([3, 4, 5])
```

Vector Operations

3.2 Vector Operations

- **Operations in Linear Algebra:**
  - **Addition and Subtraction of Vectors:**
    - $\mathbf{v} = [2, 4, 6]$
    - $\mathbf{w} = [1, 0, 1]$
    - **Addition:** $\mathbf{v} + \mathbf{w} = [2 + 1, 4 + 0, 6 + 1] = [3, 4, 7]$
    - **Subtraction:** $\mathbf{v} - \mathbf{w} = [2 - 1, 4 - 0, 6 - 1] = [1, 4, 5]$
  - **Scalar Multiplication:**
    - Scalar $c = 3$
    - $c \cdot \mathbf{v} = 3 \cdot [2, 4, 6] = [6, 12, 18]$

NumPy Implementation:

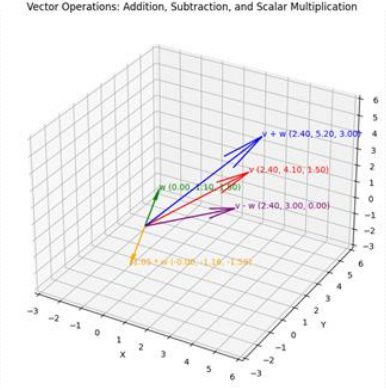
- **Performing Vector Addition and Subtraction:**

```python
import numpy as np
v = np.array([2, 4, 6])
w = np.array([1, 0, 1])
addition = v + w  # Vector addition
subtraction = v - w  # Vector subtraction
```

- **Scalar Multiplication:**

```python
scalar = 3
scaled_vector = v * scalar  # Scalar multiplication
```

Vector Operations: Addition, Subtraction, and Scalar Multiplication



3D Vector Operations Sum | Subtraction

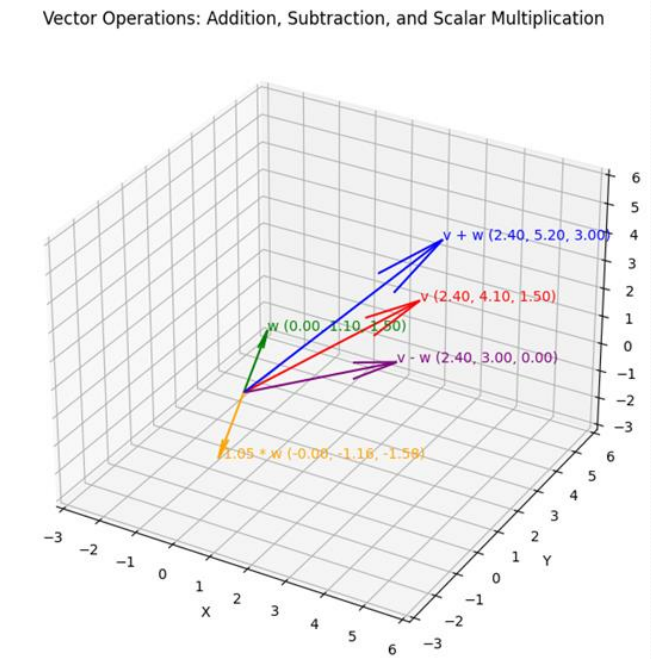
NumPy Implementation:

- **Performing Vector Addition and Subtraction:**

```python
import numpy as np
v = np.array([2, 4, 6])
w = np.array([1, 0, 1])
addition = v + w  # Vector addition
subtraction = v - w  # Vector subtraction
```

- **Scalar Multiplication:**

```python
scalar = 3
scaled_vector = v * scalar  # Scalar multiplication
```



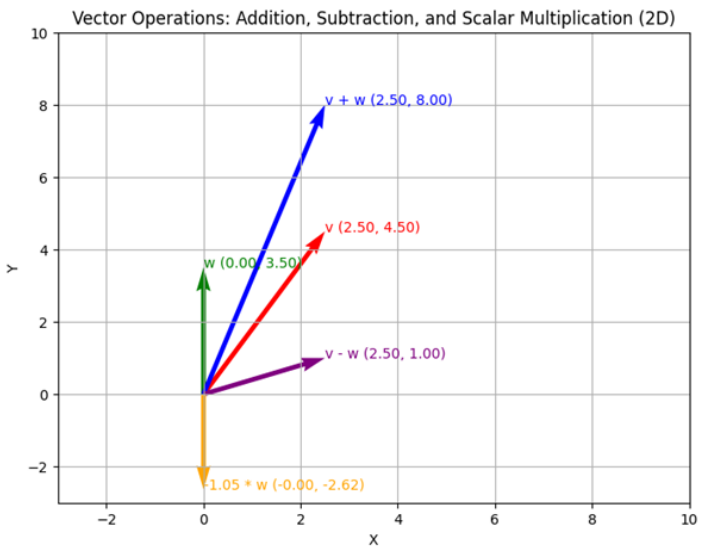
2D Vector Operations - Sum | Subtraction

```python
import numpy as np
import matplotlib.pyplot as plt

# Define the 2D vectors v and w
v = np.array([2.5, 4.5])
w = np.array([0.0, 3.5])

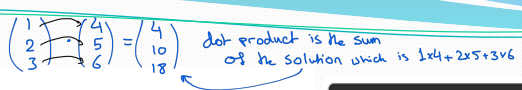
# Vector operations
addition = v + w
subtraction = v - w
c = -0.75
scalar_multiplication = c * w

```



Similarity assessment using dot product

Dot Product



- **Definition**:
  - The dot product of two vectors $\mathbf{x}$ and $\mathbf{y}$ is a scalar that represents the sum of the products of their corresponding components.
- **Compact Math Form**:
  - $\mathbf{x} \cdot \mathbf{y} = \sum_{i=1}^{n} x_i y_i$
- **Expanded Math Form**:
  - Given vectors $\mathbf{x} = [x_1, x_2, x_3]$ and $\mathbf{y} = [y_1, y_2, y_3]$, the dot product can be expressed as:

$$\mathbf{x} \cdot \mathbf{y} = \begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix} \cdot \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = x_1 \cdot y_1 + x_2 \cdot y_2 + x_3 \cdot y_3$$

- **Numerical Example**:
  - For vectors $\mathbf{x} = [2, 3, 4]$ and $\mathbf{y} = [1, 0, -1]$:

$$\mathbf{x} \cdot \mathbf{y} = 2 \cdot 1 + 3 \cdot 0 + 4 \cdot (-1) = 2 + 0 - 4 = -2$$

- **Purpose**:
  - The dot product quantifies how much one vector extends in the direction of another, thus providing a measure of alignment between the vectors. It is widely used to measure vector similarity in various fields of mathematics and physics.

python — Copy code

```python
import numpy as np

# Define the vectors
x = np.array([2, 3, 4])
y = np.array([1, 0, -1])

# Compute the dot product using NumPy
dot_product = np.dot(x, y)

# Print the results
print("Vectors x and y:")
print("x =", x)
print("y =", y)
print("Dot Product (x · y) =", dot_product)
```

```
Vectors x and y:
x = [2 3 4]
y = [ 1  0 -1]
Dot Product (x · y) = -2
```

Dot Product

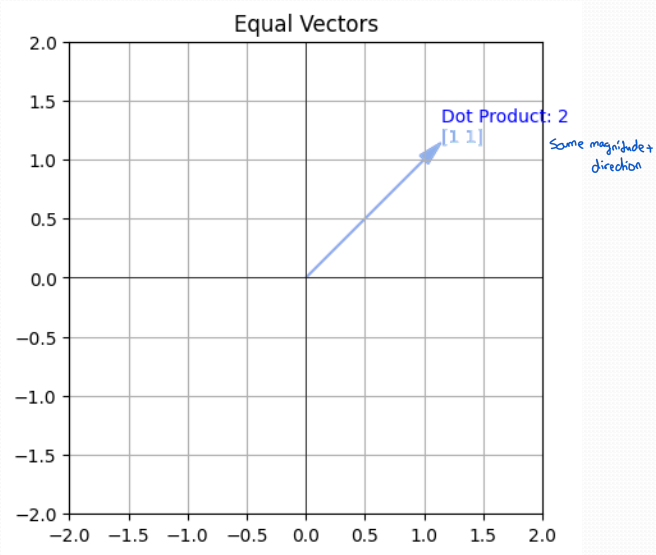
- **Linear Algebra Concept:**
  - **Dot Product**: The dot product of two vectors, feature vector $\mathbf{x}$ and weight vector $\mathbf{w}$, is calculated as $\mathbf{x} \cdot \mathbf{w} = \sum_{i=1}^{n} x_i w_i$. This operation can be seen as a linear transformation of the features by the weights.
    - **Properties**:
      - **Commutative**: $\mathbf{x} \cdot \mathbf{w} = \mathbf{w} \cdot \mathbf{x}$
      - **Distributive over addition**
      - **Scalar multiplication compatibility**

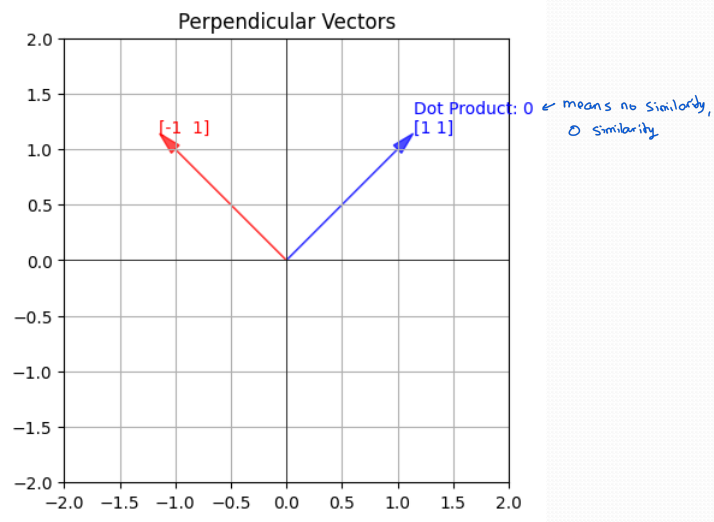
NumPy Implementation:

- **Computing Dot Product:**

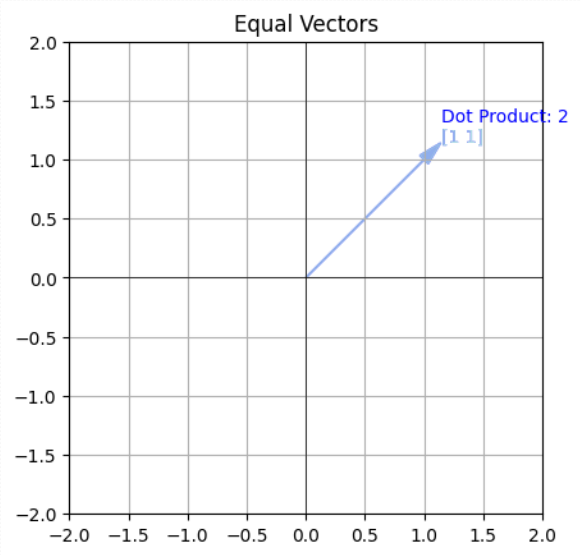
```python
import numpy as np
x = np.array([2, 4, 6])  # Feature vector
w = np.array([1, 0, 1])  # Weight vector
dot_product = np.dot(x, w)  # Dot product using np.dot
dot_product_operator = x @ w  # Dot product using @ operator
```

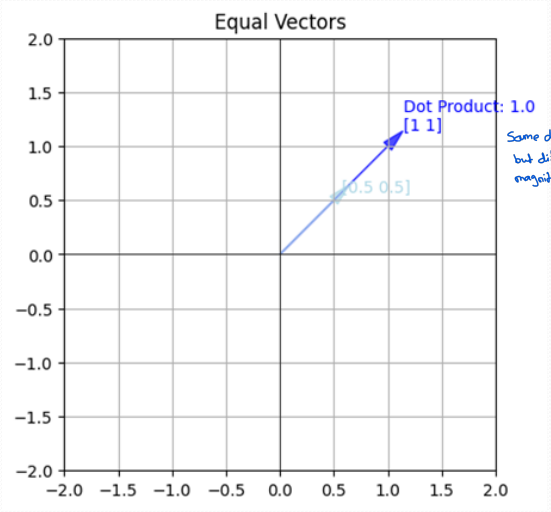
Why do We need Dot Product?



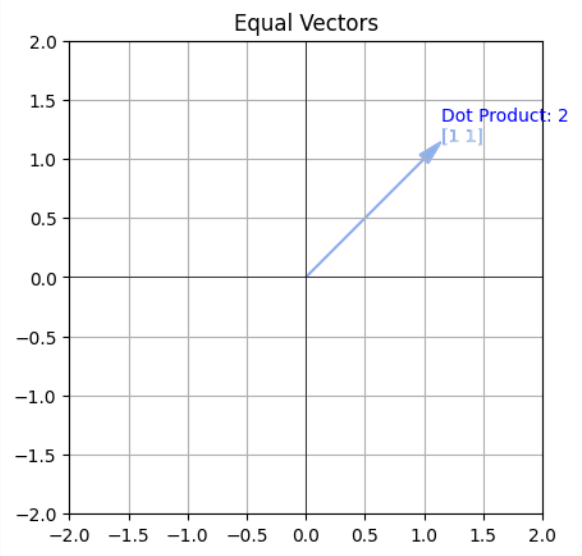


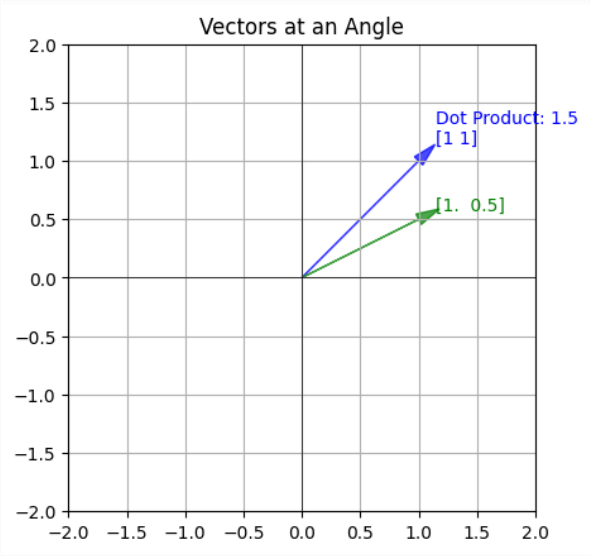
Why do We need Dot Product?



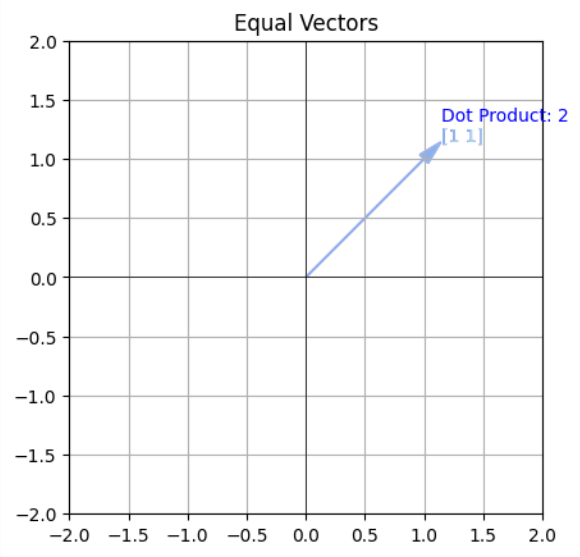


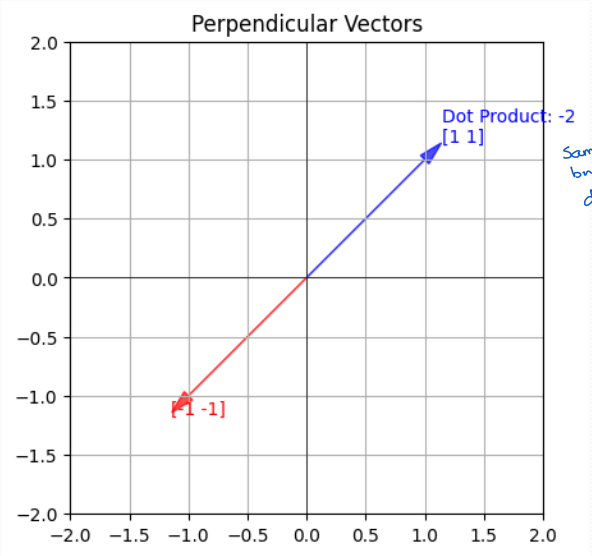
Why do We need Dot Product?





Why do We need Dot Product?





Why do We need Dot Product?

```
Dot product (similarity) between 'apple' and 'banana': 911.4376831054688
Dot product (similarity) between 'apple' and 'lion': 161.2523651123047
Embedding Shape: (300,)
```

```python
import spacy
import numpy as np

# Load the medium English model in spaCy for better embeddings
nlp = spacy.load("en_core_web_md")

# Define the words
words = ["apple", "banana", "lion"]

# Get the embeddings for the words
embeddings = {word: nlp(word).vector for word in words}

# Display the embedding vectors
for word, vector in embeddings.items():
    print(f"Embedding vector for '{word}':")
    print(vector, "\n")

# Calculate the dot product between 'apple' and 'banana', and 'apple' and 'lion'
similarity_apple_banana = np.dot(embeddings['apple'], embeddings['banana'])
similarity_apple_lion = np.dot(embeddings['apple'], embeddings['lion'])
                                                    returns a vector

# Print the results
print(f"Dot product (similarity) between 'apple' and 'banana': {similarity_apple_banana}")
print(f"Dot product (similarity) between 'apple' and 'lion': {similarity_apple_lion}")

```

Why do We need Dot Product?

```
Dot product (similarity) between 'apple' and 'banana': 39.10390090942383
Dot product (similarity) between 'apple' and 'lion': 35.11163330078125
Embedding Shape: (96,)
```

```python
import spacy
import numpy as np

# Load the medium English model in spaCy for better embeddings
nlp = spacy.load("en_core_web_sm")

# Define the words
words = ["apple", "banana", "lion"]

# Get the embeddings for the words
embeddings = {word: nlp(word).vector for word in words}

# Display the embedding vectors
for word, vector in embeddings.items():
    print(f"Embedding vector for '{word}':")
    print(vector, "\n")

# Calculate the dot product between 'apple' and 'banana', and 'apple' and 'lion'
similarity_apple_banana = np.dot(embeddings['apple'], embeddings['banana'])
similarity_apple_lion = np.dot(embeddings['apple'], embeddings['lion'])

# Print the results
print(f"Dot product (similarity) between 'apple' and 'banana': {similarity_apple_banana}")
print(f"Dot product (similarity) between 'apple' and 'lion': {similarity_apple_lion}")
print(f"Embedding Shape: {embeddings['apple'].shape}")

```

Vectors & Numpy Arrays

Vector Normalization

Vector Normalization (L2)

- **Definition**:
  - Vector normalization is the process of scaling a vector to have a unit length, i.e., a magnitude of 1. This is accomplished by dividing each component of the vector by its norm (magnitude).
- **Compact Math Form**:
  - The normalized vector $\hat{\mathbf{v}}$ of a vector $\mathbf{v}$ is given by:

$$\hat{\mathbf{v}} = \frac{\mathbf{v}}{\|\mathbf{v}\|}$$

- **Expanded Math Form**:
  - Given a vector $\mathbf{v} = [v_1, v_2, \ldots, v_n]$, its normalized form $\hat{\mathbf{v}}$ is:

$$\hat{\mathbf{v}} = \left[\frac{v_1}{\sqrt{v_1^2 + v_2^2 + \ldots + v_n^2}}, \frac{v_2}{\sqrt{v_1^2 + v_2^2 + \ldots + v_n^2}}, \ldots, \frac{v_n}{\sqrt{v_1^2 + v_2^2 + \ldots + v_n^2}} \right]$$

- **Purpose**:
  - **Similarity of Two Vectors**:
    - Normalization is crucial for methods like cosine similarity, where the similarity between two vectors is measured based on the angle between them, rather than their magnitude. By normalizing vectors, we ensure that the similarity measure reflects directional similarity, independent of vector length.

Vector Normalization – Example

- **Expanded Math Form:**
  - For vector $\mathbf{v} = [3, 4]$:

norm is square root of each value squared

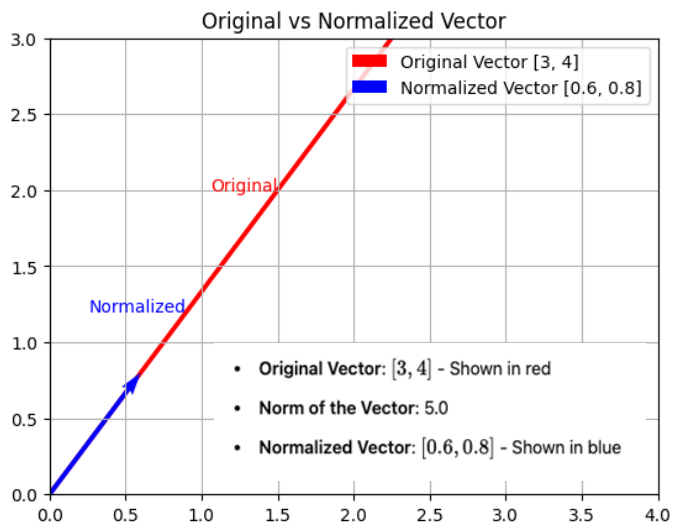
$$\|\mathbf{v}\| = \sqrt{3^2 + 4^2} = 5$$

$$\hat{\mathbf{v}} = \left[\frac{3}{5}, \frac{4}{5}\right]$$

divide each vector by the norm

- **Numerical Example:**
  - Normalize $\mathbf{v} = [3, 4]$.
    - Norm $\|\mathbf{v}\| = 5$.
    - Normalized Vector $\hat{\mathbf{v}} = [0.6, 0.8]$.
- **NumPy Implementation:**

```python
import numpy as np
v = np.array([3, 4])
norm_v = np.linalg.norm(v)
normalized_v = v / norm_v
print("Original Vector:", v)
print("Norm of v:", norm_v)
print("Normalized Vector:", normalized_v)
```



L1-Normalization

L1 Norm (Manhattan Distance):

Sum of all the absolute value of each value (to find the norm)

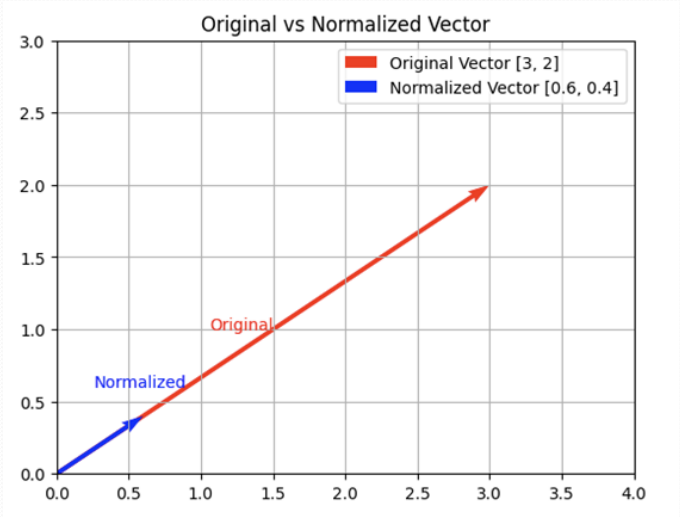
- **Compact Form:**

$$\|\mathbf{x}\|_1 = \sum_{i=1}^{n} |x_i|$$

- **Expanded Form:**
  - For vector $\mathbf{x} = [x_1, x_2, \ldots, x_n]$:

$$\|\mathbf{x}\|_1 = |x_1| + |x_2| + \ldots + |x_n|$$

- **Purpose**: Measures the sum of the absolute values of the components. It is useful in scenarios where differences of any size are equally important.



L2-Normalization

L2 Norm (Euclidean Distance):

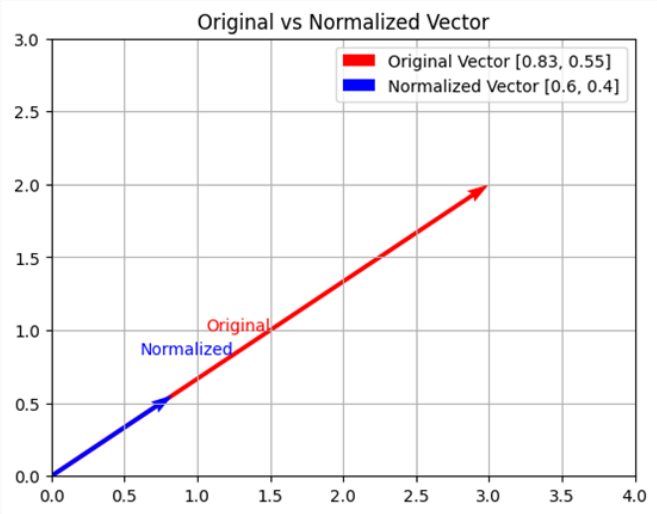
- **Compact Form:**

$$\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^{n} x_i^2}$$

- **Expanded Form:**
  - For vector $\mathbf{x} = [x_1, x_2, \ldots, x_n]$:

$$\|\mathbf{x}\|_2 = \sqrt{x_1^2 + x_2^2 + \ldots + x_n^2}$$

- **Purpose:** Measures the straight-line distance from the origin to the point in n-dimensional space, which is useful for determining actual distances between points.



Normalization in Computer Vision

```python
import numpy as np
import matplotlib.pyplot as plt
import requests
from PIL import Image
from io import BytesIO

# Load the image from a URL
image_url = "https://www.riotu-lab.org/cs313/cat.jpg"
response = requests.get(image_url)
img = Image.open(BytesIO(response.content))

# Convert the image to a numpy array
image = np.array(img)

# Convert the image to grayscale for simplicity
if image.ndim == 3:
    grayscale_image = np.dot(image[..., :3], [0.2989, 0.5870, 0.1140])
else:
    grayscale_image = image  # if the image is already in grayscale

# Flatten the grayscale image to simulate a vector of pixel intensities
image_vector = grayscale_image.flatten()

# Compute the L2 norm of the image vector
l2_norm = np.linalg.norm(image_vector)

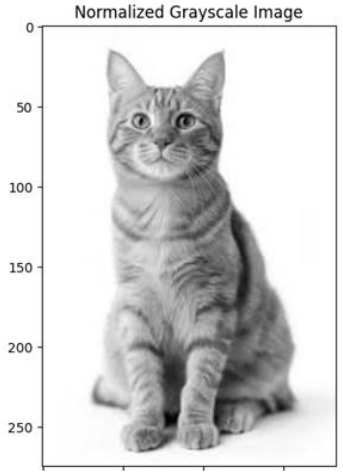
# Normalize the image vector
normalized_image_vector = image_vector / l2_norm

# Reshape normalized image back to the original grayscale image dimensions
normalized_image = normalized_image_vector.reshape(grayscale_image.shape)

# Plotting the original and normalized images
plt.figure(figsize=(12, 6))

# Plot original image
plt.subplot(1, 2, 1)
plt.imshow(grayscale_image, cmap='gray')
plt.title("Original Grayscale Image")
```






```
1  print(normalized_image_vector)
```

```
array([0.00524478, 0.00524478, 0.00524478, ..., 0.00524478, 0.00524478,
       0.00524478])
[0.00524478 0.00524478 0.00524478 ... 0.00524478 0.00524478 0.00524478]
```

```
[43]  1  print(image_vector)
```

```
[254.9745 254.9745 254.9745 ... 254.9745 254.9745 254.9745]
```

Normalization in NLP

```python
import spacy
import numpy as np

# Load the medium English model in spaCy for word embeddings
nlp = spacy.load("en_core_web_md")

# Define the words
words = ["apple", "banana", "lion"]

# Get the embeddings for the words
embeddings = {word: nlp(word).vector for word in words}

# Display the embedding vectors and calculate their norms (L2 norm)
for word, vector in embeddings.items():
    # Compute the L2 norm (Euclidean norm) of the word embedding
    norm = np.linalg.norm(vector)
    print(f"Vector norm (L2) for '{word}': {norm}")

```

```
Vector norm (L2) for 'apple': 43.36647415161133
Vector norm (L2) for 'banana': 31.6203556060791
Vector norm (L2) for 'lion': 55.14573287963867
```

Normalization in NLP

```python
import spacy
import numpy as np

# Load the medium English model in spaCy for word embeddings
nlp = spacy.load("en_core_web_md")

# Define the words
words = ["apple", "banana", "lion"]

# Get the embeddings for the words
embeddings = {word: nlp(word).vector for word in words}

# Function to normalize a vector
def normalize_vector(vector):
    norm = np.linalg.norm(vector)  # Calculate the L2 norm
    if norm == 0:  # To prevent division by zero
        return vector
    return vector / norm

# Normalize the embeddings and show the result
for word, vector in embeddings.items():
    normalized_vector = normalize_vector(vector)

    # Show original and normalized vector norms
    print(f"Original L2 norm for '{word}': {np.linalg.norm(vector)}")
    print(f"Normalized L2 norm for '{word}': {np.linalg.norm(normalized_vector)}")
    print(f"Normalized vector for '{word}':\n{normalized_vector}\n")

```

```
1.0
Original L2 norm for 'apple': 43.36647415161133
Normalized L2 norm for 'apple': 1.0
Normalized vector for 'apple':
[-0.02325298 -0.0468288  -0.0148006   0.06209405  0.00732617 -0.06148067
 -0.08617717  0.12616658 -0.11933412  0.01198114  0.12657934 -0.04823542
 -0.05749142  0.05338686  0.00906207 -0.05597181  0.07896883 -0.06115323
  0.05843454 -0.11089443 -0.04526769  0.11309428 -0.05865822 -0.03650285
 -0.08719639 -0.05750064 -0.05594414  0.08798733  0.01064832  0.02833064
 -0.00483484 -0.03240522 -0.0555429  -0.05133689  0.0328249  -0.04312087
  0.05970511  0.07801418  0.06552527 -0.03466042  0.02447513  0.03105394
  0.01761545  0.02147581  0.04973427  0.05345143  0.03137677 -0.00476566
  0.06601644  0.01765949  0.0379579   0.01013041  0.0476451  -0.05680425
 -0.06277199  0.0271869   0.00095576  0.05129308  0.04127843 -0.01273749
  0.06190266  0.08541851 -0.01359553 -0.00046061 -0.03268423 -0.0099736
 -0.04702942 -0.12144865  0.01810177  0.00215099 -0.01777687  0.04564355
 -0.02931758 -0.01038959  0.05871356  0.0341093  -0.10125333 -0.06029081
  0.05877351 -0.03483567  0.00474906  0.0393207  -0.00729227 -0.01800561
  0.02562809  0.10181367  0.12546559  0.10126255 -0.06089958  0.02357812
 -0.00602862  0.07747921  0.06238921 -0.04965356  0.0086131  -0.00793977
  0.11659697 -0.06637616 -0.0374921  -0.07797268 -0.00067771 -0.03695481
```



Vectors & Numpy Arrays

Cosine Similarity

Cosine Similarity of Two Vectors

- **Definition:**
  - Cosine similarity is a measure of similarity between two non-zero vectors within an inner product space that measures the cosine of the angle between them.
- **Math Equation:**
  - **Compact Form:**

$$\text{Cosine Similarity}(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\|\|\mathbf{y}\|}$$

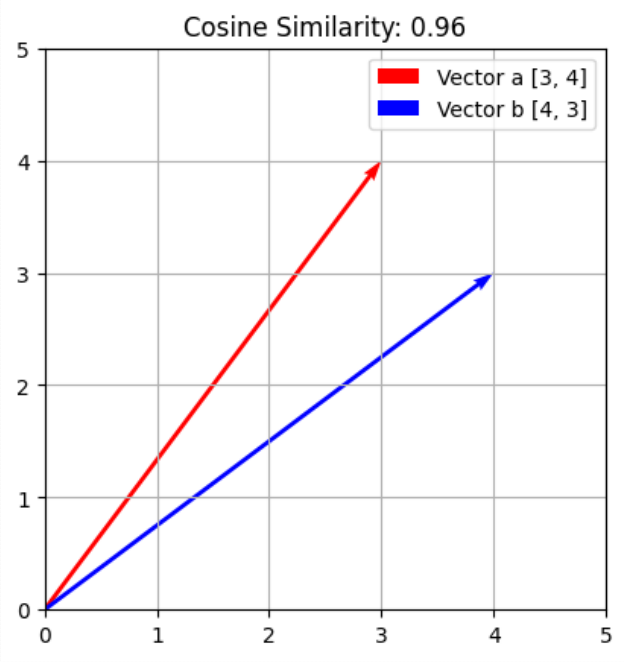
dot product of x and y

norms of x • norm of y

  - **Expanded Form:**
    - Given vectors $\mathbf{x} = [x_1, x_2, \ldots, x_n]$ and $\mathbf{y} = [y_1, y_2, \ldots, y_n]$:

$$\text{Cosine Similarity} = \frac{x_1 y_1 + x_2 y_2 + \ldots + x_n y_n}{\sqrt{x_1^2 + x_2^2 + \ldots + x_n^2} \cdot \sqrt{y_1^2 + y_2^2 + \ldots + y_n^2}}$$

- **Purpose:**
  - **Similarity Measurement:**
    - Cosine similarity is widely used in various applications such as text analysis, where it helps in identifying the similarity between documents. It is especially useful in high-dimensional positive spaces like text data in TF-IDF representation.
    - In machine learning, it helps in clustering and classification by measuring how similar the data objects are, irrespective of their magnitude.



Cosine Similarity of Two Vectors

```python
import numpy as np
import matplotlib.pyplot as plt

# Define the vectors
a = np.array([3, 4])
b = np.array([4, 3])

# Calculate the dot product
dot_product = np.dot(a, b)

# Calculate the norms of the vectors
norm_a = np.linalg.norm(a)
norm_b = np.linalg.norm(b)

# Calculate the cosine similarity
cosine_similarity = dot_product / (norm_a * norm_b)

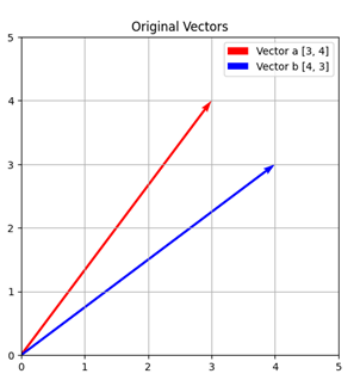
print("Cosine Similarity:", cosine_similarity)
```

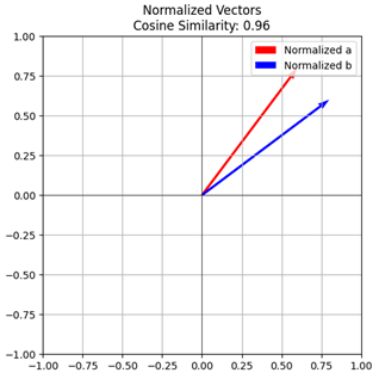
Numerical Example:

Consider two vectors in a 2D space:

- $\mathbf{a} = [3, 4]$
- $\mathbf{b} = [4, 3]$

$$\text{Cosine Similarity}(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}$$





Cosine Similarity of Two Vectors

```python
import spacy
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Load the medium English model in spaCy for word embeddings
nlp = spacy.load("en_core_web_lg")

# Define the words
words = ["apple", "banana", "fruit", "lion", "tiger", "leopard"]

# Get the embeddings for the words
embeddings = {word: nlp(word).vector for word in words}

# Function to normalize a vector
def normalize_vector(vector):
    norm = np.linalg.norm(vector)  # Calculate the L2 norm
    if norm == 0:  # To prevent division by zero
        return vector
    return vector / norm

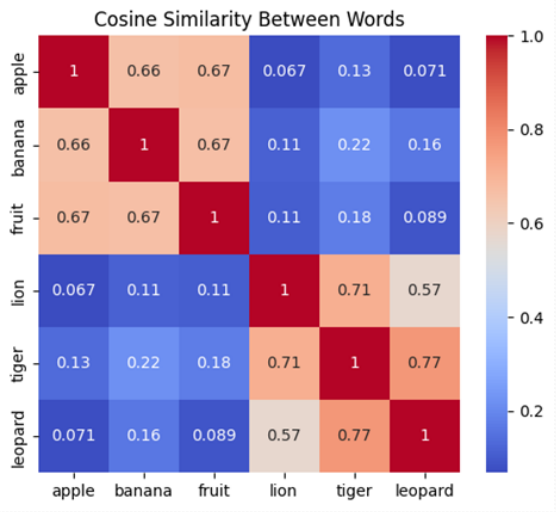
# Normalize the embeddings  to get the norm of the vector
normalized_embeddings = {word: normalize_vector(vector) for word, vector in embeddings.items()}

# Calculate pairwise cosine similarities
cosine_similarities = np.zeros((len(words), len(words)))

for i, word1 in enumerate(words):
    for j, word2 in enumerate(words):
        cosine_similarities[i, j] = np.dot(normalized_embeddings[word1], normalized_embeddings[word2])

# Plotting the cosine similarities using a heatmap
plt.figure(figsize=(6, 5))
sns.heatmap(cosine_similarities, annot=True, cmap='coolwarm', xticklabels=words, yticklabels=words)
plt.title('Cosine Similarity Between Words')
plt.show()

```



The Impact of Normalization

```python
import spacy
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Load the medium English model in spaCy for word embeddings
nlp = spacy.load("en_core_web_lg")

# Define the words
words = ["apple", "banana", "fruit", "lion", "tiger", "leopard"]

# Get the embeddings for the words
embeddings = {word: nlp(word).vector for word in words}

# Function to normalize a vector
def normalize_vector(vector):
    norm = np.linalg.norm(vector)  # Calculate the L2 norm
    if norm == 0:  # To prevent division by zero
        return vector
    return vector / norm

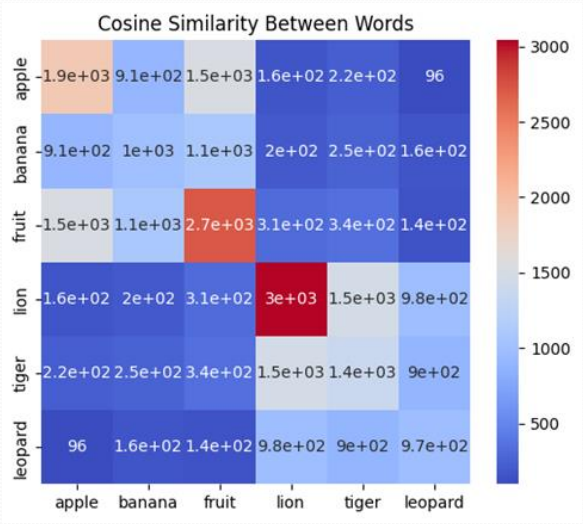
# Normalize the embeddings
normalized_embeddings = {word: normalize_vector(vector) for word, vector in embeddings.items()}

# Calculate pairwise cosine similarities
similarities = np.zeros((len(words), len(words)))

for i, word1 in enumerate(words):
    for j, word2 in enumerate(words):
        similarities[i, j] = np.dot(embeddings[word1], embeddings[word2])

# Plotting the cosine similarities using a heatmap
plt.figure(figsize=(6, 5))
sns.heatmap(similarities, annot=True, cmap='coolwarm', xticklabels=words, yticklabels=words)
plt.title('Cosine Similarity Between Words')
plt.show()
```

Not possible to compare without normalizing



The numbers are huge bcoz we didnt normalize